



**COLLEGE OF COMPUTING AND INFORMATICS
PUTRAJAYA CAMPUS**

ASSIGNMENT PART 3

SEMESTER 2 2025/2026

SUBJECT CODE : CSEB5223
SUBJECT : SOFTWARE CONSTRUCTION & METHODS
DUE DATE : 25 March 2026
NUMBER OF GROUP : GROUP 3
NAME OF MEMBERS :

No.	Full Name	Student ID
1.	AHMAD ILYAS BIN AHMAD IRFAN	SW01083514
2.	WAN NUR SABRINA BINTI BOKHARI	BSW01085928
3.	NUR HIDAYAH BINTI SARWANI	BSW01085937

Table of Contents

1.0 INTRODUCTION	8
1.1 PROJECT BACKGROUND	8
1.2 PROJECT OBJECTIVE	9
2.0 USE CASE AND CLASS DESIGN OF SLMS	11
2.1 USE CASE DIAGRAM.....	11
2.1.1 USE CASE DIAGRAM DESCRIPTION.....	11
2.1.2 ACTORS DESCRIPTION.....	14
2.1.2.1 STUDENT	14
2.1.2.1 LECTURER.....	14
2.1.2.3 ADMIN.....	15
2.1.3 SYSTEM BOUNDARY	15
2.1.4 MAIN INTERACTIONS IN THE SYSTEM.....	16
2.1.4.1 STUDENTS INTERACTION	16
2.1.4.2 LECTURERS INTERACTION	16
2.1.4.3 ADMIN INTERACTION	17
2.2 CLASS DIAGRAM.....	18
.....	18
3.0 DOCUMENTATION FORMATTING	20
4.0 CODING FILE FORMATTING	21
5.0 NAMING CONVENTIONS.....	22
6.0 FUNCTION AND METHODS MODULARITY	23
7.0 CONTINUOUS INTEGRATION RULES.....	24
8.0 CLASS DESIGN	25
8.1 CLASS DECLARATION.....	25
8.2 ATTRIBUTES	25
.....	25
8.3 CONSTRUCTOR	25
8.4 GETTERS AND SETTERS	26
.....	26
8.5 DISPLAY METHOD.....	26
9.0 COURSE INPUT IMPLEMENTATION	27
9.1 INPUT FIELDS FOR EACH COURSE ATTRIBUTES	27
9.2 ‘SUBMIT’ BUTTON TO ADD COURSE PROFILE.....	28
9.3 STORE MULTIPLE COURSES IN ARRAY/LIST	29
9.4 PREVENT ARRAY OUT-OF-BOUND ERRORS.....	30

10.0 SEARCH FUNCTION DEVELOPMENT	31
10.1 IMPLEMENT A LINEAR SEARCH FUNCTION.....	31
10.2 INPUT SEARCH CRITERIA.....	31
10.3 DISPLAY SEARCH RESULT FOR SUCCESFUL SEARCH.....	32
10.4 DISPLAY FOR UNSUCCESSFUL SEARCH	33
11.0 EDIT FUNCTION DEVELOPMENT.....	34
11.1 IMPLEMENT SEARCH FUNCTION TO FINE THE TARGET COURSE..	34
11.2 DISPLAY COMPLETE COURSE ATTRIBUTES (EDITABLE EXCEPT ..	35
COURSE CODE).....	35
11.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH.....	35
11.4 DISPLAY EDITED COURSE TO VALIDATE CHANGES	36
12.0 DELETE FUNCTION DEVELOPMENT.....	37
12.1 IMPLEMENT SEARCH FUNCTION TO FIND TARGET COURSE	37
12.2 DISPLAY COURSE ATTRIBUTES WITH CONFIRMATION MESSAGE	38
12.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH.....	39
13.0 DATA DISPLAY FUNCTION DEVELOPMENT.....	40
13.1 IMPLEMENT A ‘VIEW ALL’ FUNCTION TO DISPLAY ALL COURSES	40
.....	40
13.2 METHOD TO DISPLAY COURSE ATTRIBUTES IN ORGANIZED	41
FORMAT.....	41
14.0 ANALYSIS AND CHALLENGES	42
14.1 ANALYSIS OF SOFTWARE CONSTRUCTION BEST PRACTICES	42
14.2 CHALLENGES FACED DURING DEVELOPMENT	43
15.0 EVALUATION CRITERIA	44
15.1 FUNCTIONALITY	44
15.2 DOCUMENTATION / CODING STANDARDS.....	44
15.3 ERROR HANDLING	45
16.0 CLASS DESIGN	46
16.1 CLASS DESIGN – STUDENT CLASS.....	46
16.2 ADD STUDENT FUNCTION	47
16.3 SEARCH STUDENT.....	48
16.4 Edit Student Function	50
16.5 Delete Student Function.....	52
16.6 View All Students	54
16.7 Extra Validation	55
16.8 Evaluation of Student Profile Module	56
17.0 APPENDIX.....	58

17.1 Course.java.....	58
17.2 Student.java.....	59
17.3 CourseManager.java	62
18.0 REFERENCES	75

1.0 INTRODUCTION

1.1 PROJECT BACKGROUND

With the rapid growth of digital technology in education, learning institutions are increasingly relying on online platforms to manage teaching and learning activities. Traditional classroom-based management methods such as manual assignment submission, physical attendance tracking, and paper-based grading are inefficient and difficult to scale.

A Student Learning Management System (SLMS) is a web-based application designed to support digital learning processes. Similar to established platforms like Moodle, an SLMS enables students, lecturers, and administrators to interact within a centralized system. It allows course management, assignment submission, grading, material distribution, and discussion forums to be handled electronically.

The proposed SLMS will serve as a simplified academic platform that manages users, courses, learning materials, assignments, submissions, and discussions. The system emphasizes proper software construction practices, clean architecture, and structured development to ensure maintainability and scalability.

This project is developed as part of the Software Construction & Maintenance course to apply theoretical knowledge into practical system design and implementation.

1.2 PROJECT OBJECTIVE

The main objective of this project is to design and develop a Student Learning Management System (SLMS) by applying proper software construction principles and structured programming practices. A Student Learning Management System is a software platform used in educational institutions to manage academic information such as courses, students, and their relationships. The system helps organize course information, maintain student records, and support academic management activities in a structured and efficient way.

In this project, the SLMS focuses on managing course profiles and student profiles, allowing users to perform operations such as creating, searching, editing, and deleting records. The system also manages the relationship between students and courses, enabling the tracking of course enrolments. The development process follows modular design, object-oriented programming principles, and proper coding standards to ensure the system is scalable, readable, and maintainable.

The specific objectives of this project are:

1. To design the overall structure of the Student Learning Management System using use case diagrams and class diagrams.
2. To develop modules that allow manual input and management of course profiles, including course name, course code, credit hour, course summary, and MS Teams link.
3. To implement a student profile management module that stores and manages student information such as student ID, name, email, and phone number.
4. To implement system functionalities such as searching, editing, deleting, and displaying records for both courses and students.
5. To manage the relationship between students and courses, allowing students to be assigned to courses and courses to maintain a list of enrolled students.
6. To apply software construction best practices, including modular programming, proper naming conventions, documentation standards, and coding standards.
7. To implement error handling and input validation to ensure the system handles invalid inputs and potential errors effectively.

8. To demonstrate the use of version control and continuous integration through Github collaboration during the development process.

Through this project, students will demonstrate their understanding of structured software planning, object-oriented design, modular programming, and disciplined software development practices in building a simplified academic management system similar to existing learning platforms such as Moodle.

2.0 USE CASE AND CLASS DESIGN OF SLMS

2.1 USE CASE DIAGRAM

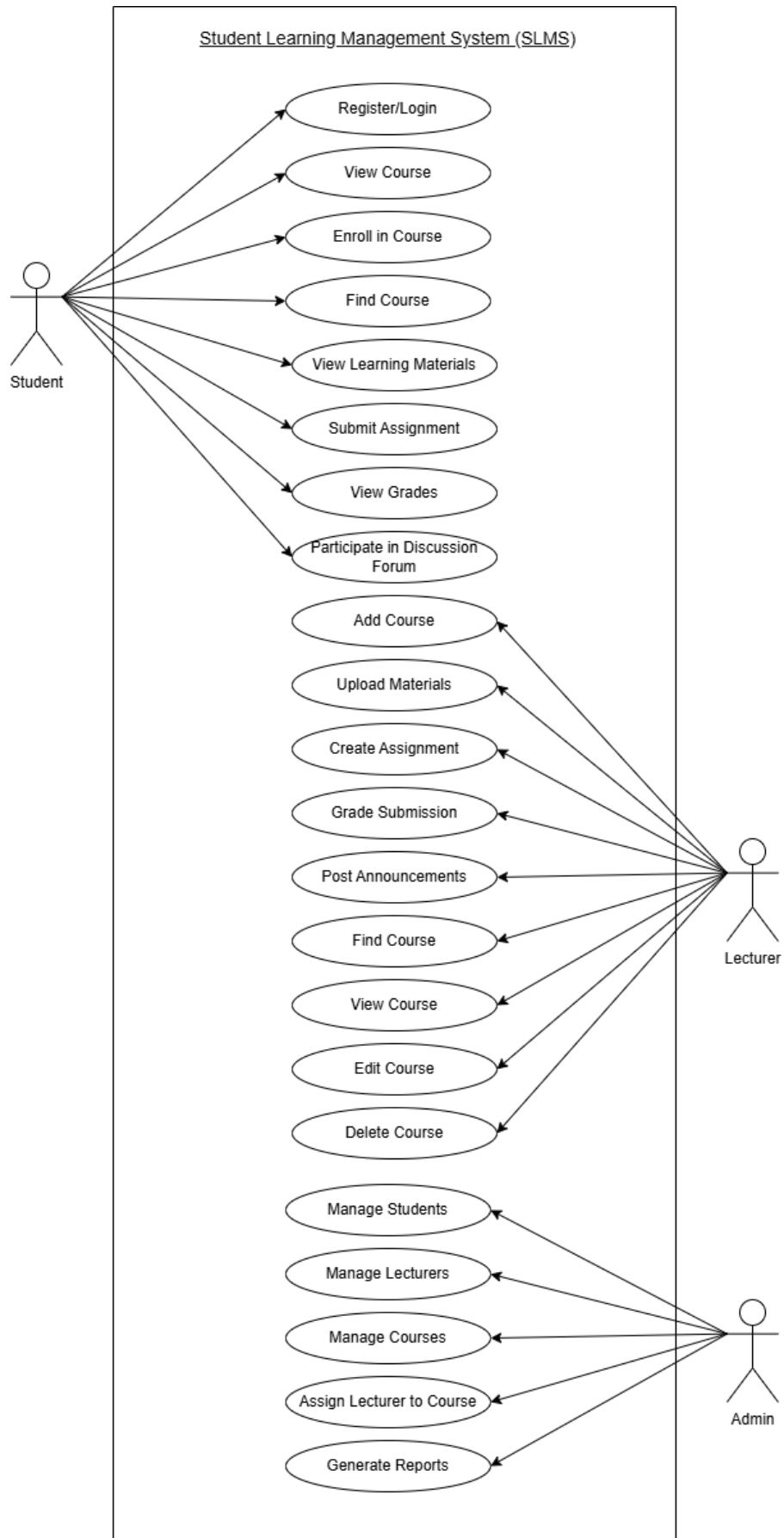
2.1.1 USE CASE DIAGRAM DESCRIPTION

A use case describes how users interact with a system to achieve a specific goal. It represents a sequence of actions between an actor and the system that leads to a useful outcome.

Use cases help developers understand:

- System functionality
- User requirements
- System interactions
- Scope of the system

Use case diagrams provide a high-level overview of the system's functionality and how external actors interact with it.



2.1.2 ACTORS DESCRIPTION

2.1.2.1 STUDENT

A Student is a user who participates in courses and learning activities. Students can access course materials, submit assignments, participate in discussions, and view grades.

Student responsibilities include:

- Register and log into the system
- Enrol in available courses
- View course
- View learning materials
- Find course
- Submit assignments
- Participate in discussion forums
- View grades

2.1.2.1 LECTURER

A Lecturer is responsible for managing courses and delivering learning content to students.

Lecturer responsibilities include:

- Add courses
- Upload learning materials
- Create assignments
- Grade student submissions
- Post announcements
- Find course
- View course
- Edit course
- Delete course

2.1.2.3 ADMIN

An Admin is responsible for managing the system and controlling overall operations.

Admin responsibilities include:

- Manage students
- Manage lecturers
- Manage courses
- Assign lecturers to courses
- Generate system reports

2.1.3 SYSTEM BOUNDARY

The system boundary represents the limits of the SLMS system. It separates the internal system functionalities from external actors.

In the use case diagram:

- All use cases are located inside the SLMS system boundary.
- Actors such as Student, Lecturer, and Admin are outside the boundary because they interact with the system.

This helps define:

- What the system is responsible for
- What actions are performed by users outside the system

2.1.4 MAIN INTERACTIONS IN THE SYSTEM

The SLMS system supports several key interactions between actors and the system.

2.1.4.1 STUDENTS INTERACTION

Students interact with the system to manage their learning activities.

Main interactions include:

- Registering and logging into the system
- Enrolling in courses
- Viewing learning materials
- Submitting assignments
- Participating in discussions
- Viewing grades
- Viewing course
- Finding course

2.1.4.2 LECTURERS INTERACTION

Lecturers interact with the system to manage course content and evaluate students.

Main interactions include:

- Adding courses
- Viewing courses
- Finding courses
- Editing courses
- Deleting courses
- Uploading learning materials
- Creating assignments
- Grading student submissions
- Providing feedback
- Posting announcements

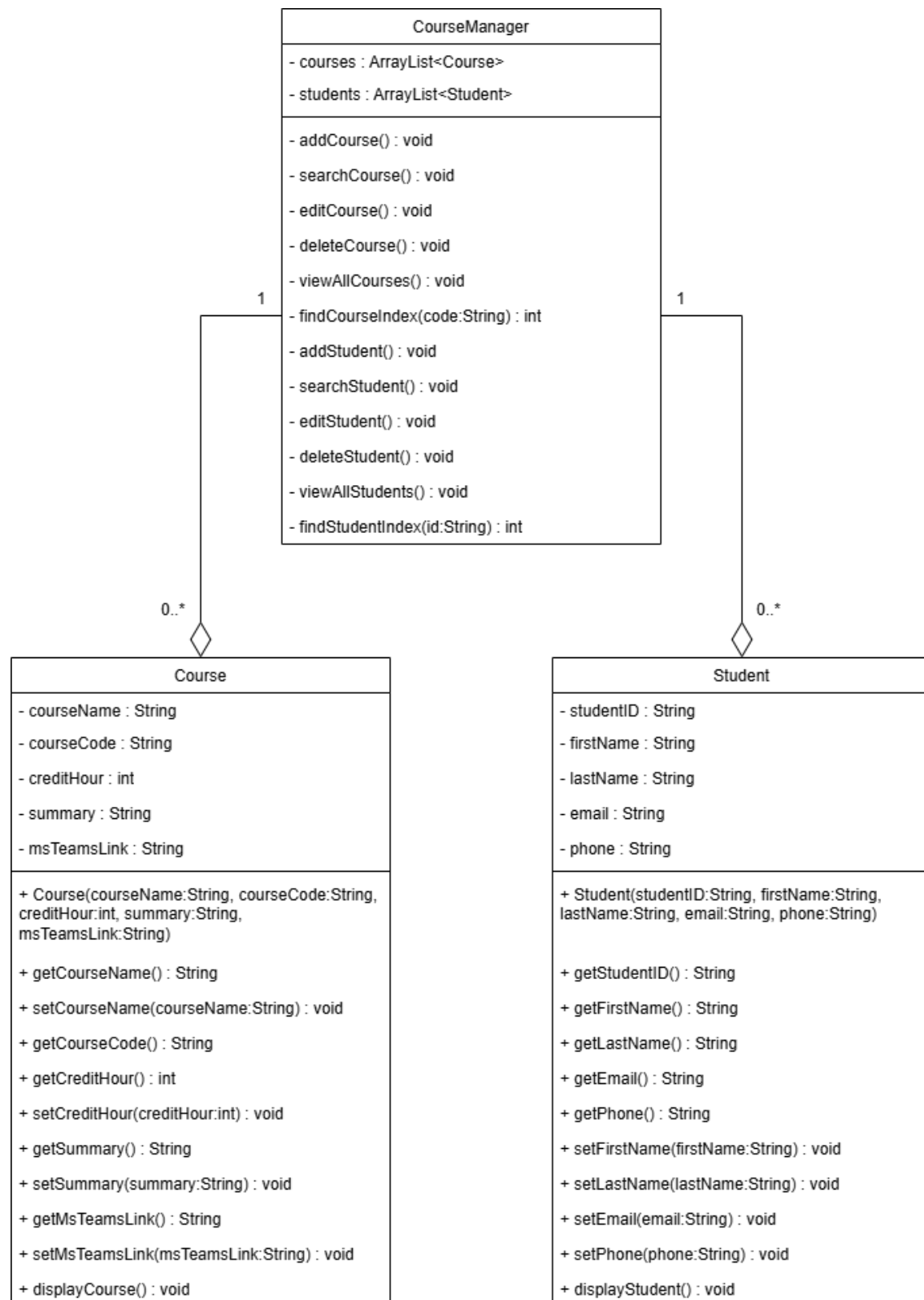
2.1.4.3 ADMIN INTERACTION

Admins interact with the system to maintain and control the platform.

Main interactions include:

- Managing students
- Managing lecturers
- Managing courses
- Assigning lecturers to courses
- Generating system reports

2.2 CLASS DIAGRAM



Object-Oriented Approach

- Used inheritance to promote reusability and reduce redundancy.

Clear Role Separation

Student, Lecturer, and Admin responsibilities are separated to enforce:

- Security
- Maintainability
- Scalability

Proper Relationship Modelling

- Association used for independent entities.
- Aggregation used when object belongs to another but can exist independently.
- Composition used when lifecycle dependency exists.

Extensibility

- Future roles (e.g., TeachingAssistant) can extend User easily.

Maintainability

- Modular class structure makes:
- Debugging easier
- Testing easier
- Feature addition safer

3.0 DOCUMENTATION FORMATTING

The SLMS documentation will follow the following format:

- Font: Times New Roman
- Font Size: 12
- Line spacing: 1.5
- Heading format:
 - 1.0 Main Heading (Bold)
 - 1.1 Subheading
- Alignment: Justified
- UML diagrams labelled properly
- Each section must include:
 - Purpose
 - Description
 - Example (if needed)

For code documentation, use:

- File header comment
- Function header comment
- Inline comments for complex logic

Example:

```
"""  
File: course.py  
Author: Team SLMS  
Description: Handles course-related operations.  
Date: 21 Feb 2026  
"""
```

4.0 CODING FILE FORMATTING

- Indentation uses 4 spaces.
- Opening and closing brackets must align.
- Comments are required before functions.
- Files organised into folders.

Example:

```
if (loginSuccess) {  
    openDashboard();  
}
```

5.0 NAMING CONVENTIONS

Element	Convention	Example
Class	PascalCase	StudentAccount
Function	camelCase	submitAssignment()
Variable	snake_case	student_name
Constant	UPPER_CASE	MAX_LOGIN_ATTEMPT
File Name	lowercase	course.py

Term	Meaning
SLMS	Student Learning Management System
LMS	Learning Management System
Enrollment	Student registered in course
Submission	Assignment uploaded by student

6.0 FUNCTION AND METHODS MODULARITY

The system is designed using a modular approach where each function and method performs a single, well-defined task. Large functionalities are broken down into smaller and reusable functions to improve readability, maintainability, and scalability of the system.

For example, user authentication is separated into different functions such as input validation, credential verification, and session handling. This modular design allows easier debugging, testing, and future enhancement of the system.

Each module is designed to minimize dependency on other modules, allowing changes to be made with minimal impact on the overall system. This approach supports team collaboration, as different members can work on different modules independently.

7.0 CONTINUOUS INTEGRATION RULES

The following continuous integration rules are implemented to ensure effective collaboration and maintain code quality throughout the development of the Student Learning Management System (SLMS):

1. Each team member must develop features in a separate feature branch.
2. Direct commits to the main branch are prohibited.
3. All changes must be reviewed and approved by the team leader before merging.
4. Clear and meaningful commit messages are required for every commit.
5. Code must be tested to ensure it functions correctly before integration.
6. GitHub is used as the main platform for version control and collaboration.

These rules help reduce integration issues and improve the overall quality of the system.

Example:

Added assignment submission module.

Fixed login validation error.

8.0 CLASS DESIGN

8.1 CLASS DECLARATION

```
3 public class Course {
```

Explanation:

- Course is the blueprint for a course.
- All course-related data and functions are organized inside this class.
- Every new course you add will be an object of this class.

8.2 ATTRIBUTES

```
4 // Attributes
5 private String courseName;
6 private String courseCode;
7 private int creditHour;
8 private String summary;
9 private String msTeamsLink;
```

Explanation:

- These are the data for each course.
- private means they are hidden from outside, to protect data integrity.
- courseCode cannot be changed after creation - this ensures uniqueness.

8.3 CONSTRUCTOR

```
11 // Constructor
12 public Course(String courseName, String courseCode, int creditHour, String summary, String msTeamsLink) {
13     this.courseName = courseName;
14     this.courseCode = courseCode;
15     this.creditHour = creditHour;
16     this.summary = summary;
17     this.msTeamsLink = msTeamsLink;
18 }
```

Explanation:

- Constructor is used to create a new course object with all required data.

8.4 GETTERS AND SETTERS

```
20 // Getters and Setters
21 public String getCourseName() { return courseName; }
22 public void setCourseName(String courseName) { this.courseName = courseName; }
23
24 public String getCourseCode() { return courseCode; }
25
26 public int getCreditHour() { return creditHour; }
27 public void setCreditHour(int creditHour) { this.creditHour = creditHour; }
28
29 public String getSummary() { return summary; }
30 public void setSummary(String summary) { this.summary = summary; }
31
32 public String getMsTeamsLink() { return msTeamsLink; }
33 public void setMsTeamsLink(String msTeamsLink) { this.msTeamsLink = msTeamsLink; }
```

Explanation:

- Getters → read/access the data.
- Setters → update/change the data (except courseCode).
- This keeps the data safe and controlled, a principle called encapsulation.

8.5 DISPLAY METHOD

```
35 // Display all course attributes
36 public void displayCourse() {
37     System.out.println("Course Name      : " + courseName);
38     System.out.println("Course Code     : " + courseCode);
39     System.out.println("Credit Hour  : " + creditHour);
40     System.out.println("Summary      : " + summary);
41     System.out.println("MS Teams Link : " + msTeamsLink);
42     System.out.println("-----");
43 }
44 }
```

Explanation:

- Method to show all course info in a clear format.
- Useful for View All, Search, Edit, and Delete functions.

9.0 COURSE INPUT IMPLEMENTATION

9.1 INPUT FIELDS FOR EACH COURSE ATTRIBUTES

```
57 private void addCourse() {
58     System.out.print("Enter Course Name: ");
59     String name = sc.nextLine();

61     String code;
62     while (true) {
63         System.out.print("Enter Course Code: ");
64         code = sc.nextLine();
65         if (findCourseIndex(code) != -1) {
66             System.out.println("Course code already exists! Enter a unique code.");
67         } else break;
68     }

70     int credit;
71     while (true) {
72         System.out.print("Enter Credit Hour: ");
73         if (sc.hasNextInt()) {
74             credit = sc.nextInt();
75             sc.nextLine();
76             if (credit > 0) break;
77             else System.out.println("Credit hour must be positive!");
78         } else {
79             System.out.println("Invalid input! Enter a number.");
80             sc.next();
81         }
82     }

84     System.out.print("Enter Course Summary: ");
85     String summary = sc.nextLine();
86
87     System.out.print("Enter MS Teams Link: ");
88     String link = sc.nextLine();

--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 1
Enter Course Name: SOFTWARE SECURITY
Enter Course Code: CSEB5113
Enter Credit Hour: 3
Enter Course Summary: SECURITY
Enter MS Teams Link: https://teams.microsoft.co
```


Explanation:

- These are the fields where the user enters course details.
- User inputs: course name, course code, credit hours, summary, and MS Teams link.

9.2 'SUBMIT' BUTTON TO ADD COURSE PROFILE

```
90 |         courses.add(new Course(name, code, credit, summary, link));  
91 |         System.out.println("Course added successfully!");  
92 |     }
```

```
Enter Course Name: SOFTWARE SECURITY  
Enter Course Code: CSEB5113  
Enter Credit Hour: 3  
Enter Course Summary: SECURITY  
Enter MS Teams Link: https://teams.mi  
Course added successfully!
```

Explanation:

- This is equivalent to a “Submit” button.
- It creates a new Course object and adds it to the ArrayList.
- The confirmation message shows that the course has been successfully added.

9.3 STORE MULTIPLE COURSES IN ARRAY/LIST

```
8      // Store multiple courses
      private ArrayList<Course> courses = new ArrayList<>();
      private Scanner sc = new Scanner(System.in);

11  /**
    --- COURSE PROFILE SYSTEM ---
    1. Add Course
    2. Search Course
    3. Edit Course
    4. Delete Course
    5. View All Courses
    0. Exit
    Choose an option: 5

    --- All Courses ---
    Course Name      : SOFTWARE SECURITY
    Course Code       : CSEB5113
    Credit Hour       : 3
    Summary           : SECURITY
    MS Teams Link     : https://teams.microsoft.com/l/t
    -----
    Course Name      : SOFTWARE CONSTRUCTION & METHOD
    Course Code       : CSEB5223
    Credit Hour       : 3
    Summary           : GITHUB AND CODING
    MS Teams Link     : https://teams.microsoft.com/l/t
    -----
```

Explanation:

- ArrayList stores all courses added by the user.
- Can store any number of courses dynamically, no fixed limit.

9.4 PREVENT ARRAY OUT-OF-BOUND ERRORS

```
101 |                                     int index = findCourseIndex(code);
102 |
103 | ☐ |                                     if (index != -1) {
104 | |                                     System.out.println("Course found:");
105 | |                                     courses.get(index).displayCourse();
106 | ☐ |                                     } else {
107 | |                                     System.out.println("Course not found.");
108 | |                                     }
109 |                                     }
```

```
--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 2
Enter Course Code to search: SCM
Course not found.
```

Explanation:

- Before accessing a course from the list, the program checks if the course exists.
- This prevents array out-of-bound errors (trying to access a non-existing index).

10.0 SEARCH FUNCTION DEVELOPMENT

10.1 IMPLEMENT A LINEAR SEARCH FUNCTION

```
205 private int findCourseIndex(String code) {  
206     for (int i = 0; i < courses.size(); i++) {  
207         if (courses.get(i).getCourseCode().equalsIgnoreCase(code)) {  
208             return i;  
209         }  
210     }  
211     return -1;  
212 }  
213 }
```

Explanation:

- This method implements a linear search algorithm.
- The program loops through the ArrayList of courses one by one.
- It compares the course code entered by the user with each course stored in the system.
- If a matching course code is found, the index of that course is returned.
- If no match is found, the method returns -1.

10.2 INPUT SEARCH CRITERIA

```
99 System.out.print("Enter Course Code to search: ");  
100 String code = sc.nextLine();  
101 int index = findCourseIndex(code);
```

```
--- COURSE PROFILE SYSTEM ---  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
0. Exit  
Choose an option: CSEB5223  
Invalid input! Enter a number: 2  
Enter Course Code to search: CSEB5223
```

Explanation:

- The user enters a course code as the search criterion.
- The program then calls the linear search method (findCourseIndex) to locate the course in the list.

10.3 DISPLAY SEARCH RESULT FOR SUCCESFUL SEARCH

```
103  if (index != -1) {  
104      System.out.println("Course found:");  
105      courses.get(index).displayCourse();  
    }
```

```
Enter Course Code to search: CSEB5223  
Course found:  
Course Name      : SCM  
Course Code      : CSEB5223  
Credit Hour     : 3  
Summary         : GITHUB  
MS Teams Link    : https://teams.microsoft.com/1/1  
-----
```

Explanation:

- If the course exists in the system, the program retrieves the course object using its index.
- The method `displayCourse()` is called to show all course attributes, including:
 - Course Name
 - Course Code
 - Credit Hour
 - Course Summary
 - MS Teams Link

10.4 DISPLAY FOR UNSUCCESSFUL SEARCH

```
106 | } else {  
107 |     System.out.println("Course not found.");  
108 | }  
109 | }
```

```
--- COURSE PROFILE SYSTEM ---  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
0. Exit  
Choose an option: 2  
Enter Course Code to search: CSEB5113  
Course not found.
```

Explanation:

- If the entered course code does not exist, the program shows a message “Course not found.”
- This ensures the system handles invalid searches properly without crashing.

11.0 EDIT FUNCTION DEVELOPMENT

11.1 IMPLEMENT SEARCH FUNCTION TO FINE THE TARGET COURSE

```
116 |         System.out.print("Enter Course Code to edit: ");
117 |         String code = sc.nextLine();
118 |         int index = findCourseIndex(code);
```

```
--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 3
Enter Course Code to edit: CSEB5223
```

Explanation:

- The system first asks the user to enter the course code of the course that needs to be edited.
- The program then calls the linear search method `findCourseIndex()` to locate the course in the `ArrayList`.
- If the course exists, its index will be returned for further editing.

11.2 DISPLAY COMPLETE COURSE ATTRIBUTES (EDITABLE EXCEPT COURSE CODE)

```
121 Course c = courses.get(index);
122 System.out.println("Editing Course: " + c.getCourseCode());
123
124 System.out.print("New Course Name: ");
125 c.setCourseName(sc.nextLine());
126
127 int credit;
128 while (true) {
129     System.out.print("New Credit Hour: ");
130     if (sc.hasNextInt()) {
131         credit = sc.nextInt();
132         sc.nextLine();
133         if (credit > 0) break;
134         else System.out.println("Credit hour must be positive!");
135     } else {
136         System.out.println("Invalid input! Enter a number.");
137         sc.next();
138     }
139 }
140 c.setCreditHour(credit);
141
142 System.out.print("New Summary: ");
143 c.setSummary(sc.nextLine());
144
145 System.out.print("New MS Teams Link: ");
146 c.setMsTeamsLink(sc.nextLine());
```

```
Editing Course: CSEB5223
New Course Name: SOFTWARE CONSTRUCTION & METHOD
New Credit Hour: 3
New Summary: GITHUB AND CODING
New MS Teams Link: https://teams.microsoft.com/1
```

Explanation:

- Once the course is found, the program retrieves the course object from the ArrayList.
- The user is allowed to edit all course attributes, including:
 - Course Name
 - Credit Hour
 - Course Summary
 - MS Teams Link
- However, the course code cannot be edited because it acts as a unique identifier for each course.

11.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH


```

150 |         } else {
151 |             System.out.println("Course not found.");
152 |         }
153 |     }

```

```

--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 3
Enter Course Code to edit: CSEB5123
Course not found.

```

Explanation:

- If the course code entered by the user does not exist, the system will display “Course not found.”
- This ensures the program handles invalid input without causing system errors.

11.4 DISPLAY EDITED COURSE TO VALIDATE CHANGES

```

148 |         System.out.println("Course updated successfully!");
149 |         c.displayCourse();

```

```

Course updated successfully!
Course Name      : SOFTWARE CONSTRUCTION & METHOD
Course Code     : CSEB5223
Credit Hour     : 3
Summary         : GITHUB AND CODING
MS Teams Link   : https://teams.microsoft.com/l/team/19'
-----

```

Explanation:

- After editing the course information, the system displays the updated course details.
- This allows the user to verify that the changes were successfully applied.

12.0 DELETE FUNCTION DEVELOPMENT

12.1 IMPLEMENT SEARCH FUNCTION TO FIND TARGET COURSE

```
160 System.out.print("Enter Course Code to delete: ");
161 String code = sc.nextLine();
162 int index = findCourseIndex(code);
```

```
--- COURSE PROFILE SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 4
Enter Course Code to delete: CSEB3123
```

Explanation:

- The system asks the user to enter the course code of the course that needs to be deleted.
- The program uses the findCourseIndex() method (linear search) to locate the course in the ArrayList.
- If the course exists, the program retrieves its index for further action.

12.2 DISPLAY COURSE ATTRIBUTES WITH CONFIRMATION MESSAGE

```
165 | | | | Course c = courses.get(index);
166 | | | | System.out.println("Course found:");
167 | | | | c.displayCourse();
168 | | | |
169 | | | | System.out.print("Confirm deletion? (Y/N): ");
170 | | | | String confirm = sc.nextLine();
```

```
Course found:
Course Name      : SOFTWARE ENGINEERING
Course Code     : CSEB3123
Credit Hour     : 3
Summary         : SDLC
MS Teams Link   : https://teams.microsof
-----
Confirm deletion? (Y/N): Y
```

Explanation:

- If the course is found, the system displays the complete course attributes using the displayCourse() method.
- This allows the user to verify the correct course before deleting it.
- The program then asks for confirmation (Y/N) before proceeding with deletion.

12.3 DISPLAY MESSAGE FOR UNSUCCESSFUL SEARCH

```
180 } else {  
181     System.out.println("Course not found.");  
182 }  
183 }
```

```
--- COURSE PROFILE SYSTEM ---  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
0. Exit  
Choose an option: 4  
Enter Course Code to delete: CSEB5113  
Course not found.
```

Explanation:

- If the entered course code does not exist, the system displays the message “Course not found.”
- This prevents the program from attempting to delete a non-existing course.

13.0 DATA DISPLAY FUNCTION DEVELOPMENT

13.1 IMPLEMENT A ‘VIEW ALL’ FUNCTION TO DISPLAY ALL COURSES

```
46 | | | | | case 5: viewAllCourses(); break;
```

```
--- COURSE PROFILE SYSTEM ---
```

```
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
0. Exit
Choose an option: 5
```

Explanation:

- The system provides a menu option (5 – View All Courses) that acts like a “View All” button.
- When the user selects this option, the program calls the viewAllCourses() method.
- This method displays all courses currently stored in the system.

13.2 METHOD TO DISPLAY COURSE ATTRIBUTES IN ORGANIZED FORMAT

```
188  private void viewAllCourses() {
189      if (courses.isEmpty()) {
190          System.out.println("No courses available.");
191      } else {
192          System.out.println("\n--- All Courses ---");
193          for (Course c : courses) {
194              c.displayCourse();
195          }
196      }
197  }
```



```
--- All Courses ---
Course Name      : SOFTWARE SECURITY
Course Code      : CSEB5113
Credit Hour     : 3
Summary         : SECURITY
MS Teams Link    : https://teams.microsoft.com/l/tea
-----
Course Name      : SOFTWARE CONSTRUCTION & METHOD
Course Code      : CSEB5223
Credit Hour     : 3
Summary         : GITHUB AND CODING
MS Teams Link    : https://teams.microsoft.com/l/tea
-----
```

Explanation:

- This method displays all courses stored in the ArrayList.
- It first checks whether the list is empty.
- If courses exist, the system loops through the list and calls displayCourse() for each course.
- The displayCourse() method ensures the information is shown in a clear and organized format.

14.0 ANALYSIS AND CHALLENGES

14.1 ANALYSIS OF SOFTWARE CONSTRUCTION BEST PRACTICES

The development of the Course Profile System follows several software construction best practices to ensure the system is organized, maintainable, and reliable. First, the program applies object-oriented programming principles by separating the system into two main classes: Course and CourseManager. The Course class is responsible for storing course attributes such as course name, course code, credit hour, course summary, and MS Teams link. The CourseManager class manages the system operations including adding, searching, editing, deleting, and displaying course profiles.

Additionally, the program uses modular programming, where each functionality is implemented in separate methods such as addCourse(), searchCourse(), editCourse(), deleteCourse(), and viewAllCourses(). This makes the program easier to understand, maintain, and debug. The use of ArrayList also allows the system to store multiple course objects dynamically without needing a fixed-size array.

14.2 CHALLENGES FACED DURING DEVELOPMENT

During the development of the Course Profile System, several challenges were encountered. One of the main challenges was implementing the linear search function to correctly locate courses using the course code. It required careful handling to ensure that the system returns the correct index or an appropriate message if the course does not exist.

Another challenge was handling user input validation. The program needed to ensure that the user enters valid data, such as numeric values for credit hours. This was handled using methods such as `hasNextInt()` to prevent invalid input from causing runtime errors.

Additionally, ensuring that duplicate course codes are not allowed was another challenge. Since the course code acts as a unique identifier, the program had to check the existing course list before allowing a new course to be added. Implementing proper checks for search, edit, and delete operations was also important to avoid accessing invalid indexes and causing array out-of-bound errors.

15.0 EVALUATION CRITERIA

15.1 FUNCTIONALITY

The Course Profile System successfully supports the required functionalities for managing course profiles. The system allows users to input course information, including course name, course code, credit hour, course summary, and MS Teams link. These course profiles are stored in an ArrayList and can be displayed using the View All function.

The system also provides features to search, edit, and delete course profiles. The search function uses a linear search algorithm to locate courses based on their course code. The edit function allows users to modify course details except for the course code, which remains unchanged as a unique identifier. The delete function allows users to remove a selected course after confirmation. These functions ensure that the system can effectively manage course information.

15.2 DOCUMENTATION / CODING STANDARDS

The program follows appropriate documentation and coding standards to improve readability and maintainability. Comments are included throughout the code to explain the purpose of classes and methods. Meaningful variable names and method names such as `addCourse()`, `searchCourse()`, and `viewAllCourses()` help make the code easier to understand.

The program structure is also well organized by separating responsibilities into different classes. The `Course` class focuses on representing course data, while the `CourseManager` class handles system operations and user interaction. This separation of responsibilities follows good software design principles.

15.3 ERROR HANDLING

The program includes several error handling techniques to ensure that the system runs smoothly even when users provide invalid input. For example, the program checks whether the user enters a valid numeric value when inputting credit hours by using `hasNextInt()`.

The system also checks whether a course code already exists before adding a new course to prevent duplicate entries. In addition, before performing search, edit, or delete operations, the program verifies whether the course exists by using the `findCourseIndex()` method. If the course is not found, the system displays an appropriate message such as “Course not found.” These mechanisms help prevent runtime errors and improve the overall reliability of the system.

16.0 CLASS DESIGN

16.1 CLASS DESIGN – STUDENT CLASS

- **Attributes:**

```
* This class includes:  
* - Constructor to create a Student object  
* - Getter methods to access attributes  
* - Setter methods to modify editable attributes  
* - displayStudent method to show student information in a clear format  
*/  
public class Student {  
  
    // Attributes  
    private String studentID; // Unique student identifier, cannot be changed after creation  
    private String firstName; // Student's first name  
    private String lastName; // Student's last name  
    private String email; // Student's email address  
    private String phone; // Student's phone number
```

- studentID (String) – Unique, immutable identifier for each student.
- firstName (String) – Student’s first name.
- lastName (String) – Student’s last name.
- email (String) – Student’s email address.
- phone (String) – Student’s contact number.

- **Methods:**

- **Getters:** Allow other classes (e.g., CourseManager) to access private attributes.
- **Setters:** Allow modification of firstName, lastName, email, phone but not studentID.
- **displayStudent():** Prints all student attributes in a clear and organized format.

16.2 ADD STUDENT FUNCTION

The system allows users to manually enter student information.

```
// Validate all inputs at once
if (id.isBlank() || fname.isBlank() || lname.isBlank() || email.isBlank() || phone.isBlank()) {
    System.out.println("Student not added! All fields must be filled.");
} else {
    // Only add if all fields are valid
    students.add(new Student(id, fname, lname, email, phone));
    System.out.println("Student added successfully!");
}
```

- User enters Student ID, First Name, Last Name, Email, and Phone Number.
- The system validates whether any field is empty.
- If any field is blank:
 - "Student not added! All fields must be filled."
- If all fields are filled:
 - "Student added successfully!"
- The student object is then stored in an **ArrayList**, allowing multiple student records to be stored dynamically.

SUCCESSFULLY ADDED:

```
--- SLMS SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Add Student
7. Search Student
8. Edit Student
9. Delete Student
10. View All Students
0. Exit
Choose an option: 6
Enter Student ID: BSW01085928
Enter First Name: Wan Nur
Enter Last Name: Sabrina
Enter Email: sabrina@gmail.com
Enter Phone: 0123456678
Student added successfully!
```

STUDENT FAIL TO FILL ALL FIELDS:

```
Choose an option: 6
Enter Student ID:
Enter First Name: wan
Enter Last Name: sabrina
Enter Email: sabrina@gmail.com
Enter Phone: 0165766482
Student not added! All fields must be filled.
```

16.3 SEARCH STUDENT

The search function allows users to find a student using **Student ID**.

```
private void searchStudent() {
    System.out.print("Enter Student ID to search: ");
    String id = sc.nextLine();

    int index = findStudentIndex(id); // search for student

    if (index != -1) {
        System.out.println("Student found:");
        students.get(index).displayStudent(); // Show student details
    } else {
        System.out.println("Student not found."); // not found
    }
}
```

- User enters a student ID.
- The system uses linear search with equalsIgnoreCase() → search is case-insensitive.
- If found → displays all student information using displayStudent().
- If not found → displays: "Student not found."
- Search is case-insensitive, so entering lowercase/uppercase IDs works.
- Example: searching for bsw01085998 successfully finds the student.

STUDENT WAS FOUND:

```
Choose an option: 6
Enter Student ID: BSW01085998
Enter First Name: Ilyas
Enter Last Name: Ahmad
Enter Email: ilyass@gmail.com
Enter Phone: 0198765543
Student added successfully!
7. Search Student
8. Edit Student
9. Delete Student
10. View All Students
0. Exit
Choose an option: 7
Enter Student ID to search: bsw01085998
Student found:
Student ID   : BSW01085998
Name        : Ilyas Ahmad
Email       : ilyass@gmail.com
Phone       : 0198765543
-----
```

STUDENT WAS NOT FOUND:

```
Choose an option: 7
Enter Student ID to search: BSW01098765
Student not found.
```

16.4 Edit Student Function

- User enters Student ID.
- The system performs a **linear search** to locate the student.

If found:

The user can edit:

- FirstName
- LastName
- Email
- phone

The **studentID cannot be edited** because it is a unique identifier.

After editing, the system displays the **updated student information** to confirm the changes.

If student is not found:

"Student not found."

BEFORE EDIT:

```
Choose an option: 6
Enter Student ID: BSW01085973
Enter First Name: Hidayah
Enter Last Name: Sarwani
Enter Email: dayah@gmail.com
Enter Phone: 0167542987
Student added successfully!
```

AFTER EDITED:

```
Choose an option: 8
Enter Student ID to edit: BSW01085973
New First Name: Nur Dayah
New Last Name: Sarwani
New Email: dayah@gmail.com
New Phone: 0167542987
Student updated successfully.
Student ID   : BSW01085973
Name        : Nur Dayah Sarwani
Email       : dayah@gmail.com
Phone       : 0167542987
-----
```

STUDENT NOT FOUND:

```
Choose an option: 8
Enter Student ID to edit: BSW01085974
Student not found.
```

Good Practice Observed:

- Clear separation of ID (immutable) and editable fields.
- Updated info displayed for verification.

16.5 Delete Student Function

```
private void deleteStudent() {
    System.out.print("Enter Student ID to delete: ");
    String id = sc.nextLine();

    int index = findStudentIndex(id);

    if (index != -1) {
        Student s = students.get(index);
        System.out.println("Student found:");
        s.displayStudent(); // show info before deletion

        System.out.print("Confirm deletion? (Y/N): ");
        String confirm = sc.nextLine();

        if (confirm.equalsIgnoreCase("Y")) {
            students.remove(index); // delete student
            System.out.println("Student deleted successfully!");
            viewAllStudents(); // Show updated list
        } else {
            System.out.println("Deletion cancelled.");
        }
    } else {
        System.out.println("Student not found.");
    }
}
```

- User enters student ID.
- If ID not found → "Student not found."
- If found → system displays student info and asks for confirmation (Y/N).
- User confirms deletion → student removed from list.

Outputs:

- **Student Found and Deleted**

```
--- SLMS SYSTEM ---
1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Add Student
7. Search Student
8. Edit Student
9. Delete Student
10. View All Students
0. Exit
Choose an option: 9
Enter Student ID to delete: BSW01085973
Student found:
Student ID   : BSW01085973
Name        : Nur Dayah Sarwani
Email       : dayah@gmail.com
Phone       : 0167542987
-----
Confirm deletion? (Y/N): Y
Student deleted successfully!

--- All Students ---
Student ID   : BSW01085928
Name        : Wan Nur Sabrina
Email       : sabrina@gmail.com
Phone       : 0123456678
-----
```

- **Student ID Not Found**

```
Enter Student ID to delete: bsw01098786
Student not found.
```

Good Practices Observed:

- Confirms deletion to prevent accidental removal.
- Case-insensitive confirmation input (y/Y) works.
- Clear feedback for both success and failure.

16.6 View All Students

The **View All Students** function displays all student records stored in the system.

Implementation:

- The system loops through the **ArrayList** of students.
- The **displayStudent()** method is called for each student.

Before displaying, the system checks whether the list is empty to prevent errors.

This ensures student information is shown in a **clear and organized format**.

```
public void displayStudent(){
    System.out.println("Student ID   : " + studentID);
    System.out.println("Name       : " + firstName + " " + lastName);
    System.out.println("Email      : " + email);
    System.out.println("Phone     : " + phone);
    System.out.println("-----");
}
```

OUTPUT:

```
--- All Students ---
Student ID   : BSW01085928
Name        : Wan Nur Sabrina
Email       : sabrina@gmail.com
Phone      : 0123456678
-----
Student ID   : BSW01085973
Name        : Hidayah Sarwani
Email       : dayah@gmail.com
Phone      : 0167542987
-----
Student ID   : BSW01085998
Name        : Ilyas Ahmad
Email       : ilyass@gmail.com
Phone      : 0198765543
-----
```

Good Practices Observed:

- **Reusability:** Uses displayStudent() instead of repeating print statements.
- **Validation:** Handles empty list gracefully.
- Checks if the list is empty.

16.7 Extra Validation

- Lowercase IDs still work due to equalsIgnoreCase().
- Example: Searching for bsw01085976 still finds the student.

LOWERCASE ID:

```
Enter Student ID to search: bsw01085998
Student found:
Student ID   : BSW01085998
Name        : Ilyas Ahmad
Email       : ilyass@gmail.com
Phone       : 0198765543
-----
```

LOWERCASE CONFIRMATION:

```
Choose an option: 9
Enter Student ID to delete: bsw01085998
Student found:
Student ID   : BSW01085998
Name        : Ilyas Irfan
Email       : ilyass@gmail.com
Phone       : 0198765563
-----
```

```
Confirm deletion? (Y/N): y
Student deleted successfully!
```

```
--- All Students ---
Student ID   : BSW01085928
Name        : Wan Nur Sabrina
Email       : sabrina@gmail.com
Phone       : 0123456678
-----
```

Good Practices:

- All fields validated after input.
- User-friendly: all fields input first without interruption.
- Clear feedback: success/failure at the end.
- Encapsulation: Student attributes remain private, added via constructor.

16.8 Evaluation of Student Profile Module

The Student Profile module successfully meets the requirements stated in Lab 5–6 evaluation criteria. The system allows users to input, search, edit, delete, and display student profiles correctly, ensuring proper functionality of all required operations. The student data is stored dynamically using an ArrayList, which prevents array out-of-bound errors and allows flexible data management.

In terms of integration, the Student Profile module is successfully combined with the Course Profile module in the CourseManager system, allowing users to manage both student and course information within a single menu-driven interface.

The program also follows good documentation and coding standards. Each class is clearly structured with proper comments explaining the purpose of attributes, constructors, and methods. Encapsulation is applied by keeping attributes private and providing getter and setter methods for controlled access.

Furthermore, the program demonstrates effective error handling and input validation. The system checks for empty fields when adding a student, prevents editing of the immutable student ID, handles cases where a student is not found during search/edit/delete operations, and confirms deletion to prevent accidental data removal. These practices improve system reliability and user experience.

Overall, the Student Profile module is well-designed, user-friendly, and satisfies the functionality, integration, documentation, and error-handling requirements specified in the lab evaluation criteria.

17.0 APPENDIX

17.1 Course.java

```
1  /**
2   * File: Course.java
3   * Author: Team SLMS (Group 3)
4   * Description: Handles individual course data and provides methods to display it.
5   * Date: 02 Mar 2026
6   */
7
8  package com.mycompany.courseprofilesyste;
9
10 /**
11  * Class: Course
12  * Description: Represents a course profile in the SLMS system.
13  * Stores course information such as name, code, credit hour, summary,
14  * and MS Teams link.
15  */
16
17 public class Course {
18     // Attributes available in the course
19     private String courseName;
20     private String courseCode; // cannot be edited after creation
21     private int creditHour;
22     private String summary;
23     private String msTeamsLink;
24
25     // Constructor for the course
26     public Course(String courseName, String courseCode, int creditHour, String summary, String msTeamsLink) {
27         this.courseName = courseName;
28         this.courseCode = courseCode;
29         this.creditHour = creditHour;
30         this.summary = summary;
31         this.msTeamsLink = msTeamsLink;
32     }
33
34     // Getters and Setters for the required course's attributes
35     public String getCourseName() { return courseName; }
36     public void setCourseName(String courseName) { this.courseName = courseName; }
37
38     public String getCourseCode() { return courseCode; }
39
40     public int getCreditHour() { return creditHour; }
41     public void setCreditHour(int creditHour) { this.creditHour = creditHour; }
42
43     public String getSummary() { return summary; }
44     public void setSummary(String summary) { this.summary = summary; }
45
46     public String getMsTeamsLink() { return msTeamsLink; }
47     public void setMsTeamsLink(String msTeamsLink) { this.msTeamsLink = msTeamsLink; }
48
49     // Display all course's attributes
50     public void displayCourse() {
51         System.out.println("Course Name : " + courseName);
52         System.out.println("Course Code : " + courseCode);
53         System.out.println("Credit Hour : " + creditHour);
54         System.out.println("Summary : " + summary);
55         System.out.println("MS Teams Link: " + msTeamsLink);
56         System.out.println("-----");
57     }
58 }
```

17.2 Student.java

```
1  /**
2   * File: Student.java
3   * Author: Team SLMS (Group 3)
4   * Description: Handles individual student data and provides methods to display it.
5   * Date: 17 Mar 2026
6   */
7
8  package com.mycompany.courseprofilesyste;
9
10 /**
11  * Class: Student
12  * Description:
13  * This class represents a Student object in the SLMS system.
14  *
15  * Each student has all these attributes:
16  * - studentID: unique identifier (cannot be edited)
17  * - firstName
18  * - lastName
19  * - email
20  * - phone
21  *
22  * This class includes:
23  * - Constructor to create a Student object
24  * - Getter methods to access attributes
25  * - Setter methods to modify editable attributes
26  * - displayStudent method to show student information in a clear format
27  */
28  public class Student {
29
30      // Student's attributes
31      private String studentID; // Unique student identifier, cannot be changed after creation
32      private String firstName; // Student's first name
33      private String lastName; // Student's last name
34      private String email; // Student's email address
35      private String phone; // Student's phone number
36
37
38      /**
39       * Constructor: Student
40       * Purpose:
41       * - Initialize a new Student object with all required attributes
42       * - studentID is passed in constructor because it is unique and must not be changed later
43       */
44      public Student(String studentID, String firstName, String lastName, String email, String phone){
45          this.studentID = studentID;
46          this.firstName = firstName;
47          this.lastName = lastName;
48          this.email = email;
49          this.phone = phone;
50      }
51
52      /**
53       * Getter methods:
54       * These allow other classes (e.g., CourseManager) to access student attributes.
55       * We do not allow direct access to private attributes for data encapsulation.
56       *
57       * Encapsulation ensures data safety and control over how attributes are accessed or modified.
58       */
```



```

59
60     public String getStudentID(){
61         return studentID; // StudentID is unique and immutable, so only getter is provided
62     }
63
64     public String getFirstName(){
65         return firstName;
66     }
67
68     public String getLastName(){
69         return lastName;
70     }
71
72     public String getEmail(){
73         return email;
74     }
75
76     public String getPhone(){
77         return phone;
78     }
79
80     /**
81      * Setter methods:
82      * These allow modification of attributes except studentID.
83      * Why no setter for studentID?
84      * - studentID is the unique identifier.
85      * - Changing it could cause duplicate conflicts or break search/delete operations.
86      */
87
88     public void setFirstName(String firstName){
89         this.firstName = firstName;
90     }
91
92     public void setLastName(String lastName){
93         this.lastName = lastName;
94     }
95
96     public void setEmail(String email){
97         this.email = email;
98     }
99
100    public void setPhone(String phone){
101        this.phone = phone;
102    }
103
104    /**
105     * Method: displayStudent
106     * Purpose:
107     * - To show all student attributes in a clear and organized format
108     * - Used after search, edit, or delete operations in CourseManager
109     *
110     * Why use a separate display method?
111     * - To avoid repeating System.out.println code in multiple places
112     * - Improves code reusability and readability
113     */
114
115    public void displayStudent(){
116        System.out.println("Student ID : " + studentID);
117        System.out.println("Name : " + firstName + " " + lastName);
118        System.out.println("Email : " + email);
119        System.out.println("Phone : " + phone);
120        System.out.println("-----");
121    }
122 }

```


17.3 CourseManager.java

```
1  /**
2   * File: CourseManager.java
3   * Author: Team SLMS (Group 3)
4   * Description: Main program to manage course profiles.
5   * Features: Add, Search, Edit, Delete, View All courses.
6   */
7
8  package com.mycompany.courseprofilesystem;
9
10 import java.util.ArrayList;
11 import java.util.Scanner;
12
13 /**
14  * Class: CourseManager
15  *
16  * This class manages both Course Profile and Student Profile modules
17  * in the SLMS (Student Learning Management System).
18  *
19  * Users can:
20  * - Add, search, edit, delete, and view courses
21  * - Add, search, edit, delete, and view students
22  *
23  * ArrayLists are used for dynamic storage of Course and Student objects.
24  * Scanner is used for user input.
25  */
26 class CourseManager {
27
28     // ArrayList to store all courses. ArrayList allows dynamic resizing.
29     private ArrayList<Course> courses = new ArrayList<>();
30
31     // ArrayList to store all students. Dynamic insertion and deletion possible.
32     private ArrayList<Student> students = new ArrayList<>();
33
34     // Scanner to get user input
35     private Scanner sc = new Scanner(System.in);
36
37     /**
38      * Main method: program entry point
39      */
40     public static void main(String[] args) {
41         CourseManager manager = new CourseManager();
42         manager.run(); // Start the interactive menu
43     }
44
45     /**
46      * Method: run()
47      *
48      * This is the main menu loop. It uses a do-while loop.
49      *
50      * Why use do-while?
51      * - Ensures menu displays at least once before checking exit.
52      * - Loop continues until user chooses 0 (Exit).
53      */
54 }
```



```

54  ✓    public void run() {
55        int choice;
56        do {
57            // Display menu options
58            System.out.println("\n--- SLMS SYSTEM ---");
59            System.out.println("1. Add Course");
60            System.out.println("2. Search Course");
61            System.out.println("3. Edit Course");
62            System.out.println("4. Delete Course");
63            System.out.println("5. View All Courses");
64            System.out.println("6. Add Student");
65            System.out.println("7. Search Student");
66            System.out.println("8. Edit Student");
67            System.out.println("9. Delete Student");
68            System.out.println("10. View All Students");
69            System.out.println("0. Exit");
70            System.out.print("Choose an option: ");
71
72            /**
73             * Input validation using while loop.
74             * - Checks if the user entered an integer.
75             * - If input is invalid (like letters), the user need to prompt again.
76             * - sc.next() clears the invalid input.
77             */
78            while (!sc.hasNextInt()) {
79                System.out.print("Invalid input! Enter a number: ");
80                sc.next();
81            }
82            choice = sc.nextInt();
83            sc.nextLine(); // consume newline
84
85            // Switch case to execute chosen operation
86            switch (choice) {
87                case 1: addCourse(); break;
88                case 2: searchCourse(); break;
89                case 3: editCourse(); break;
90                case 4: deleteCourse(); break;
91                case 5: viewAllCourses(); break;
92                case 6: addStudent(); break;
93                case 7: searchStudent(); break;
94                case 8: editStudent(); break;
95                case 9: deleteStudent(); break;
96                case 10: viewAllStudents(); break;
97                case 0: System.out.println("Exiting program."); break;
98                default: System.out.println("Invalid choice!");
99            }
100
101        } while (choice != 0); // Loop until user enters 0
102    }

```

```

104     /**
105      * Method: addCourse()
106      * Adds a new course after valid inputs.
107      */
108     private void addCourse() {
109         System.out.print("Enter Course Name: ");
110         String name = sc.nextLine();
111
112         String code;
113
114         // Ensure unique course code
115         while (true) {
116             System.out.print("Enter Course Code: ");
117             code = sc.nextLine();
118
119             // findCourseIndex() searches if code already exists
120             if (findCourseIndex(code) != -1) {
121                 System.out.println("Course code already exists! Enter another unique code.");
122             } else break; // Exit loop when course code is unique
123         }
124
125         int credit;
126         // Validate credit hour input: integer and > 0
127         while (true) {
128             System.out.print("Enter Credit Hour: ");
129             if (sc.hasNextInt()) {
130                 credit = sc.nextInt();
131                 sc.nextLine();
132                 if (credit > 0) break;
133                 else System.out.println("Credit hour must be positive!");
134             } else {
135                 System.out.println("Invalid input! Enter a number.");
136                 sc.next();
137             }
138         }
139
140         System.out.print("Enter Course Summary: ");
141         String summary = sc.nextLine();
142
143         System.out.print("Enter MS Teams Link: ");
144         String link = sc.nextLine();
145
146         // Validate all inputs at once
147         if (name.isBlank() || code.isBlank() || credit.isBlank() || summary.isBlank() || link.isBlank()) {
148             System.out.println("Course not added! All fields must be filled.");
149         } else {
150             // Only add if all fields are valid
151             courses.add(new Course(name, code, credit, summary, link));
152             System.out.println("Course added successfully!");
153         }

```

```

151     /**
152     * Method: searchCourse()
153     * Search for a course by course code.
154     */
155     ✓ private void searchCourse() {
156         System.out.print("Enter Course Code to search: ");
157         String code = sc.nextLine();
158
159         int index = findCourseIndex(code);
160
161         if (index != -1) {
162             System.out.println("Course found:");
163             courses.get(index).displayCourse(); // Display course info
164         } else {
165             System.out.println("Course not found.");
166         }
167     }
168

```

```

169      /**
170       * Method: editCourse()
171       * Edit existing course details (except code).
172       */
173  ✓ private void editCourse() {
174      System.out.print("Enter Course Code to edit: ");
175      String code = sc.nextLine();
176      int index = findCourseIndex(code);
177
178      if (index != -1) {
179          Course c = courses.get(index);
180          System.out.println("Editing Course: " + c.getCourseCode());
181
182          System.out.print("New Course Name: ");
183          c.setCourseName(sc.nextLine());
184
185          int credit;
186          // Validate input to new credit hour: integer and > 0
187          while (true) {
188              System.out.print("New Credit Hour: ");
189              if (sc.hasNextInt()) {
190                  credit = sc.nextInt();
191                  sc.nextLine();
192                  if (credit > 0) break;
193                  else System.out.println("Credit hour must be positive!");
194              } else {
195                  System.out.println("Invalid input! Enter a number.");
196                  sc.next();
197              }
198          }
199          c.setCreditHour(credit);
200
201          System.out.print("New Summary: ");
202          c.setSummary(sc.nextLine());
203
204          System.out.print("New MS Teams Link: ");
205          c.setMsTeamsLink(sc.nextLine());
206
207          System.out.println("Course updated successfully!");
208          c.displayCourse();
209      } else {
210          System.out.println("Course not found.");
211      }
212  }
213

```



```

219      /**
220       * Method: deleteCourse()
221       * Deletes a course after confirmation by the user.
222       */
223      private void deleteCourse() {
224          System.out.print("Enter Course Code to delete: ");
225          String code = sc.nextLine();
226          int index = findCourseIndex(code);
227
228          //check if the course code entered exists or not.
229          if (index != -1) {
230              Course c = courses.get(index);
231              System.out.println("Course found:");
232              c.displayCourse();
233
234              //confirmation to user to delete, must be (y/Y) to delete
235              System.out.print("Confirm deletion? (Y/N): ");
236              String confirm = sc.nextLine();
237              if (confirm.equalsIgnoreCase("Y")) {
238                  courses.remove(index); // Remove from list
239                  System.out.println("Course deleted successfully!");
240              } else {
241                  System.out.println("Deletion cancelled.");
242              }
243          } else {
244              System.out.println("Course not found.");
245          }
246      }
247
248      /**
249       * Method: viewAllCourses()
250       * Displays all courses in the system.
251       */
252      private void viewAllCourses() {
253          /**check for all courses created and courses available in the database.
254           * if empty/none available, display the following message
255           * if courses exists, display all courses created.
256           */
257          if (courses.isEmpty()) {
258              System.out.println("No courses available.");
259          } else {
260              System.out.println("\n--- All Courses ---");
261              for (Course c : courses) {
262                  c.displayCourse();
263              }
264          }
265      }
266
267      /**
268       * Method: findCourseIndex()
269       * Searches course list for a code. Returns index or -1 if not found.
270       */
271      private int findCourseIndex(String code) {
272          for (int i = 0; i < courses.size(); i++) {
273              // equalsIgnoreCase: ignores uppercase/lowercase differences
274              if (courses.get(i).getCourseCode().equalsIgnoreCase(code)) {
275                  return i; // Course found

```



```

280      /**
281       * Method: addStudent()
282       * Description: Adds a new student to the system
283       * - Input all details from user
284       * - Stores Student object in ArrayList
285       */
286  ✓ private void addStudent() {
287      // Ask for all inputs first
288      System.out.print("Enter Student ID: ");
289      String id = sc.nextLine();
290
291      System.out.print("Enter First Name: ");
292      String fname = sc.nextLine();
293
294      System.out.print("Enter Last Name: ");
295      String lname = sc.nextLine();
296
297      System.out.print("Enter Email: ");
298      String email = sc.nextLine();
299
300      System.out.print("Enter Phone: ");
301      String phone = sc.nextLine();
302
303      // Validate all inputs at once
304      if (id.isBlank() || fname.isBlank() || lname.isBlank() || email.isBlank() || phone.isBlank()) {
305          System.out.println("Student not added! All fields must be filled.");
306      } else {
307          // Only add if all fields are valid
308          students.add(new Student(id, fname, lname, email, phone));
309          System.out.println("Student added successfully!");
310      }
311  }
312
313      /**
314       * Method: searchStudent()
315       * Description: Search student by ID
316       * - Uses findStudentIndex()
317       * - Display the student's information if found
318       */
319  ✓ private void searchStudent() {
320      System.out.print("Enter Student ID to search: ");
321      String id = sc.nextLine();
322
323      int index = findStudentIndex(id); // search for student
324
325      if (index != -1) {
326          System.out.println("Student found:");
327          students.get(index).displayStudent(); // display student details
328      } else {
329          System.out.println("Student not found."); // not found
330      }
331  }

```

```

332     /**
333      * Method: editStudent()
334      * - Updates student information by ID
335      * - Prompts user to enter new data
336      * - Ensures correct student is updated using index
337      */
338     private void editStudent() {
339         System.out.print("Enter Student ID to edit: ");
340         String id = sc.nextLine();
341
342         int index = findStudentIndex(id);
343
344         //if id entered exists in database, proceed to update student details
345         if (index != -1) {
346             Student s = students.get(index);
347
348             System.out.print("New First Name: ");
349             s.setFirstName(sc.nextLine());
350
351             System.out.print("New Last Name: ");
352             s.setLastName(sc.nextLine());
353
354             System.out.print("New Email: ");
355             s.setEmail(sc.nextLine());
356
357             System.out.print("New Phone: ");
358             s.setPhone(sc.nextLine());
359
360             System.out.println("Student updated successfully.");
361             s.displayStudent();
362         } else {
363             //if id entered does not exists, display the error message
364             System.out.println("Student not found.");
365         }
366     }
367

```

```

368      /**
369       * Method: deleteStudent()
370       * - Find student index first
371       * - Confirm deletion
372       * - Remove from ArrayList
373       * - Display all students after deletion
374       */
375  ✓ private void deleteStudent() {
376      System.out.print("Enter Student ID to delete: ");
377      String id = sc.nextLine();
378
379      int index = findStudentIndex(id);
380
381      //if id entered exists in database, proceed to update student details
382      if (index != -1) {
383          Student s = students.get(index);
384          System.out.println("Student found:");
385          s.displayStudent(); // display student's information before deletion
386
387          //ask confirmation from user to delete, must be (y/Y)
388          System.out.print("Confirm deletion? (Y/N): ");
389          String confirm = sc.nextLine();
390
391          if (confirm.equalsIgnoreCase("Y")) {
392              students.remove(index); // delete student
393              System.out.println("Student deleted successfully!");
394              viewAllStudents(); // Show updated list
395          } else {
396              System.out.println("Deletion cancelled.");
397          }
398      } else {
399          //if id entered does not exists, display the error message
400          System.out.println("Student not found.");
401      }
402  }
403
404      /**
405       * Method: viewAllStudents()
406       * - Loops through ArrayList and display each student
407       * - Checks if list is empty first
408       */
409  ✓ private void viewAllStudents() {
410      /**check for all students available in the database.
411       * if empty/none available, display the following message
412       * if students exists, display all students' information.
413       */
414      if (students.isEmpty()) {
415          System.out.println("No students available.");
416      } else {
417          System.out.println("\n--- All Students ---");
418          for (Student s : students) {
419              s.displayStudent();
420          }
421      }
422  }

```



```

424      /**
425       * Method: findStudentIndex()
426       * - Returns index of student in ArrayList
427       * - Linear search
428       * - Returns -1 if not found
429       * - equalsIgnoreCase() to ignore case sensitivity in ID
430       */
431  ✓ private int findStudentIndex(String id) {
432      for (int i = 0; i < students.size(); i++) {
433          // equalsIgnoreCase: allows case-insensitive comparison
434          if (students.get(i).getStudentID().equalsIgnoreCase(id)) {
435              return i; // student found, display student's information
436          }
437      }
438      // -1 indicates student not found
439      return -1;
440  }
441  }

```

18.0 REFERENCES

Moodle Official Website. <https://moodle.org/>

Moodle Codebase Repository. <https://github.com/moodle/moodle>